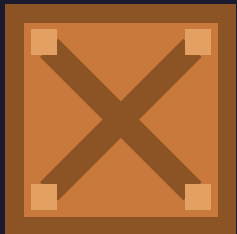


SIA TP1

Nicolás Rivas - Lautaro Bonseñor - Juan Ignacio Causse
Javier Emmanuel Liu - Augusto Andrés Lanari



Parte 1.

8-Puzzle

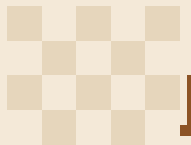


Estructura de Estado - Tupla de 9 enteros

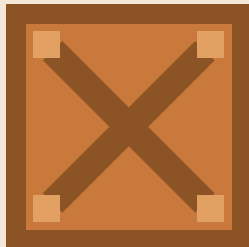
Ventajas por sobre una matriz 3×3:

- ❑ Es inmutable
- ❑ Se puede usar como clave de un `set` o `hashmap`
- ❑ Permite detectar estados repetidos eficientemente

1	6	5
7	3	8
	4	2



Heurísticas Admisibles

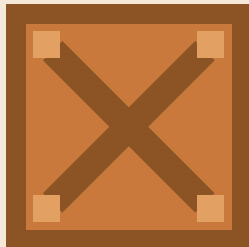


Heurística 1 - Distancia Manhattan

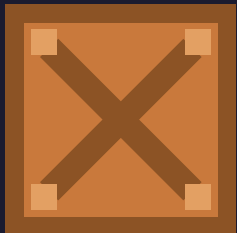
Heurística 2 - Fichas "Fuera de lugar"



Método de Búsqueda: A* con Distancia Manhattan

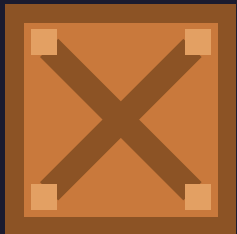


- ❑ **DFS** no garantiza optimalidad
- ❑ **IDDFS** consume menos memoria, pero no aprovecha nuestras heurísticas
- ❑ **BFS** es una posibilidad, pero no utiliza información
- ❑ **A*** garantiza que la solución sea óptima con una heurística admisible



Parte 2.

Sokoban



Movimiento del Jugador vs Movimiento de Cajas

Estructura de Estados



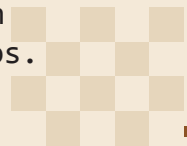
Estructura de Estados

La estructura que se le da a los estados resulta muy importante debido a que de ella dependen:

- ❑ La capacidad de embeber los mismos en estructuras de datos estándar de Python.
- ❑ La conveniencia de su uso a la hora de generar una API estándar sobre la que implementar todos los algoritmos de búsqueda.
- ❑ La robustez y eficiencia generales de la implementación.

Teniendo esto en consideración, se tomaron las siguientes decisiones:

- ❑ Los **estados** deben ser **inmutables por naturaleza**, pues:
 - ❑ Proveen mayor rapidez,
 - ❑ Menor posibilidad de errores, y
 - ❑ Capacidad de **hashear** los propios estados para poder usarse en *diccionarios* o *sets*.
- ❑ Debe haber un **buen diseño de clases**, ya que la modularización ordena el código y permite el **buen uso de la orientación a objetos**.
- ❑ Debido a que cada **estado difiere de sus hijos en un movimiento**, se le dio a los propios **estados** la **posibilidad de generar a sus sucesores**, es decir, los tableros proveen un método que permite generar sus tableros derivados y obtener un iterador de los mismos.
- ❑ Debe hacerse un **profundo uso de typing**.





Definición Original



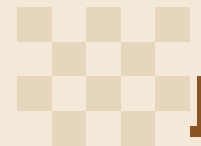
Por movimiento del jugador

Estado **estático**:

- ❑ Muros
- ❑ Ubicación de los objetivos

Estado **dinámico**:

- ❑ Posición del jugador
- ❑ Posición de las cajas





Coordinate y Direction

Clases `Coordinate` y `Direction` expresan la posición actual de los objetos y las direcciones de movimiento, respectivamente. Permiten aritmética entre ellos, haciendo más expresiva la API de estados.

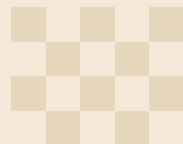
```
class Coordinate(NamedTuple): 57+ usages  @jcausse
    """Immutable ``(row, col)`` position on a 2-D grid.

    Extends :class:`~typing.NamedTuple`, so instances are lightweight, hashable,
    and support tuple unpacking::

        pos = Coordinate(2, 5)
        row, col = pos          # unpacking
        {pos}                   # usable in sets / dict keys
        pos + Direction.UP.value # arithmetic with direction deltas

    :param row: Row index (0-indexed, grows downward).
    :param col: Column index (0-indexed, grows rightward).
    """
```

```
class Direction(Enum):
    UP = (-1, 0)
    DOWN = (1, 0)
    LEFT = (0, -1)
    RIGHT = (0, 1)
```





ImmutableMatrix

Cada tablero toma como base una **matriz conformada por tuplas** (debido a que las listas son `Unhashable Types`), cuya dimensión se determina de forma dinámica a la hora de cargar el nivel a resolver.

```
class ImmutableMatrix(Generic[_T]): 8+ usages  Ⓙjcausse
    """Immutable, tuple-backed 2-D matrix.

    Every row is a ``tuple[_T, ...]`` and the matrix itself is a
    ``tuple[tuple[_T, ...], ...]``, so the entire structure is deeply
    immutable and hashable.

    Use :class:`ImmutableMatrixBuilder` to construct instances
    incrementally.

    :param matrix: A tuple of equal-length row tuples.
    :raise TypeError: If *matrix* is not a tuple of tuples.
    :raise ValueError: If rows have inconsistent lengths.
    """

    __slots__ = ("_rows", "_cols", "_matrix", "_hash")
```



Board

Se utilizó una clase abstracta (paquete `ABC`) para representar un tablero genérico, que establece cuáles son los métodos de interés para las implementaciones de los algoritmos.

```
class Board(ABC): 34+ usages  Ⓙjcausse
    """Abstract base class for board-game states used with search algorithms."""

    @abstractmethod  Ⓙjcausse
    def move(self, direction: Direction) -> Self | None:
        """Apply a move, returning a new state or ``None`` if illegal."""

    @property 1 usage  Ⓙjcausse
    @abstractmethod
    def is_solved(self) -> bool:
        """Whether this state satisfies the goal condition."""

    @abstractmethod 1 usage  Ⓙjcausse
    def is_deadlock(self) -> bool:
        """Whether this state is provably unsolvable."""

    @abstractmethod 2+ usages  Ⓙjcausse
    def derived_boards(self) -> Iterator[tuple[Direction, Self]]:
        """Generator that yields tuples of ``(direction, board)`` for each board that can be derived from the current
        board by performing a move in a certain :class:`Direction`."""
```



SokobanBoard

Clase que extiende `Board` para proveer una implementación concreta del tablero de Sokoban.

```
class SokobanBoard(Board): 16+ usages 吕jcausse+1
    """Immutable Sokoban game state.

    The board separates *static* terrain (walls, goals, floor) stored in an
    :class:`ImmutableMatrix[Tile]` from *dynamic* entities (player position
    and box positions). Two boards are equal iff their player position, box
    positions **and** grid are identical; the hash is computed accordingly.

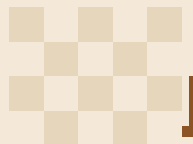
    :param grid: The static terrain matrix.
    :param player_position: Current player :class:`Coordinate`.
    :param box_positions: Iterable of box :class:`Coordinate` values.

    :raise ValueError: If positions are out of bounds, on walls, or duplicated.
    """

    __slots__ = (
        "_grid",
        "_player_position",
        "_box_positions",
        "_goal_coordinates",
        "_deadlock_positions",
        "_hash",
    )
```

```
@override 吕jcausse
def derived_boards(self) -> Iterator[tuple[Direction, SokobanBoard]]:
    """Yield all reachable successor states with the move that produced them.

    :return: ``(direction, new_board)`` pairs for every legal move.
    """
    for direction in Direction:
        successor = self.move(direction)
        if successor is not None:
            yield direction, successor
```





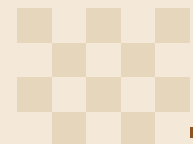
Node

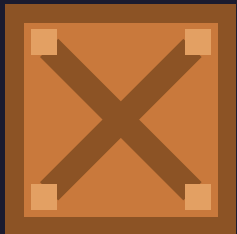
Implementación del nodo utilizado para los algoritmos de búsqueda.

```
class Node: 21+ usages  @jcausse +1

    __slots__ = ("_board", "_depth", "_parent", "_direction")

    def __init__(self, @jcausse
        board: Board,
        depth: int = 0,
        parent: Optional[Node] = None,
        direction: Optional[Direction] = None
    ) -> None:
        self._board: Board = board
        self._depth: int = depth
        self._parent: Optional[Node] = parent
        self._direction: Optional[Direction] = direction
```





Dead Square - Freeze - Corral

Deadlocks



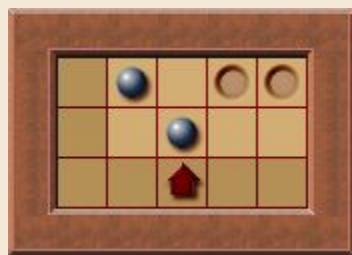
Deadlocks

Dead
Square



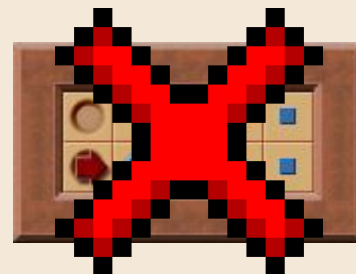
Estático

Freeze



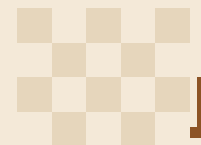
Dinámico

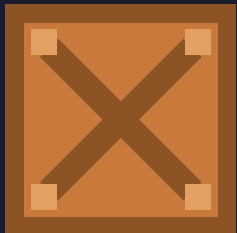
Corral



Requiere un Solver

http://sokobano.de/wiki/index.php?title=How_to_detect_deadlocks



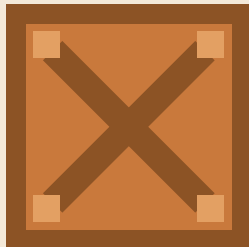


Distancia Player-to-Box vs Boxes-to-Goals

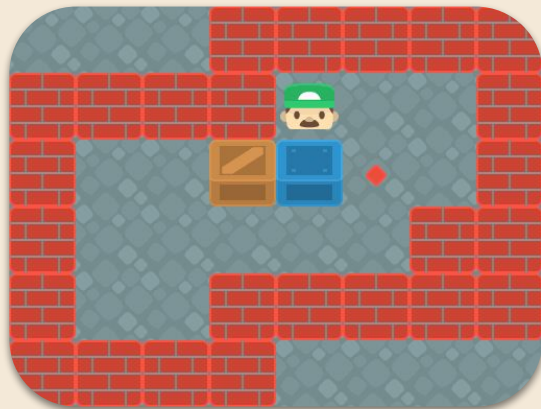
Heurísticas

Heurística 1. Distancia Mínima Box-to-Goal

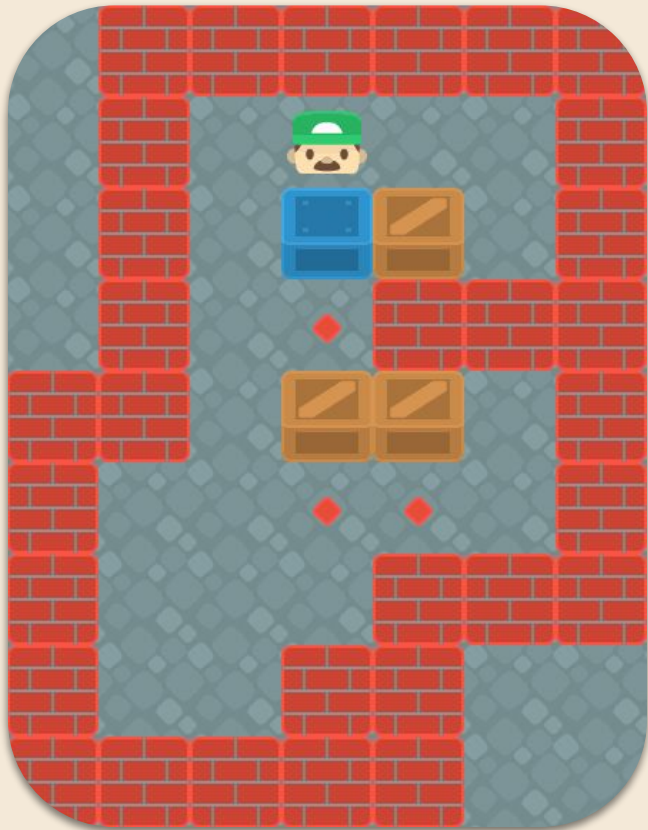
La distancia mínima entre cada caja y algún objetivo



- ❑ Asume que **no existen** colisiones entre cajas o muros y que muchas cajas **pueden** ir a un **único objetivo**
- ❑ No hay sobreestimación, pues relaja el problema y no bloquea metas



Heurística 2. Algoritmo Húngaro

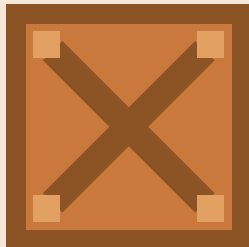


Similar a la primera heurística, pero sin repetir objetivos

- ❑ Mucho más costosa ($O(n^3)$), pero reduce la cantidad de nodos explorados
- ❑ La librería scipy utiliza una versión optimizada con el algoritmo LAPJV.

Heurística NA1. $H1 + \text{Distancia Player-To-Box}$

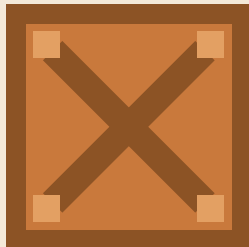
La primera heurística, pero teniendo en cuenta la distancia mínima entre una caja y el jugador



- ❑ Hace un doble conteo de pasos
- ❑ Distancia Manhattan no es un camino válido
 - ❑ Requiere que haga un rodeo para situarse en el lado opuesto

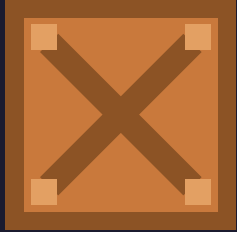
Heurística NA2. Algoritmo Húngaro Greedy

Similar a la idea de Algoritmo Húngaro, pero con “asignación miope”



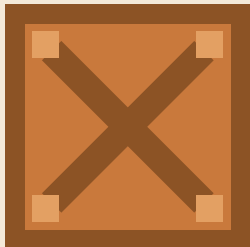
- ❑ No explora todas las combinaciones posibles (Probabilidad de sobreestimación)
- ❑ Es más rápida ($O(N*M)$)

DFS vs BFS vs A* vs Greedy BFS



Métodos de Búsqueda: Resultados

Recolección de Datos



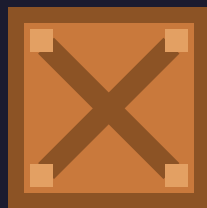
```
#####  
####  #  
#### $. #  
# $. ##  
# @ #####  
# ####  
####
```

Uso de sokobano.de para la búsqueda de niveles preexistentes

- ❑ Parseo de archivos .sok [Sokoban YASC](http://Sokoban.YASC) a niveles individuales
- ❑ El sitio provee la solución óptima

Todos los niveles utilizados en la recolección de datos fueron recolectados de la colección [Microban IV](http://Microban.IV)

- ❑ 102 niveles
- ❑ Lista curada de 56 niveles que pasan
- ❑ Rango amplio de soluciones de 30 a 25.000 movimientos

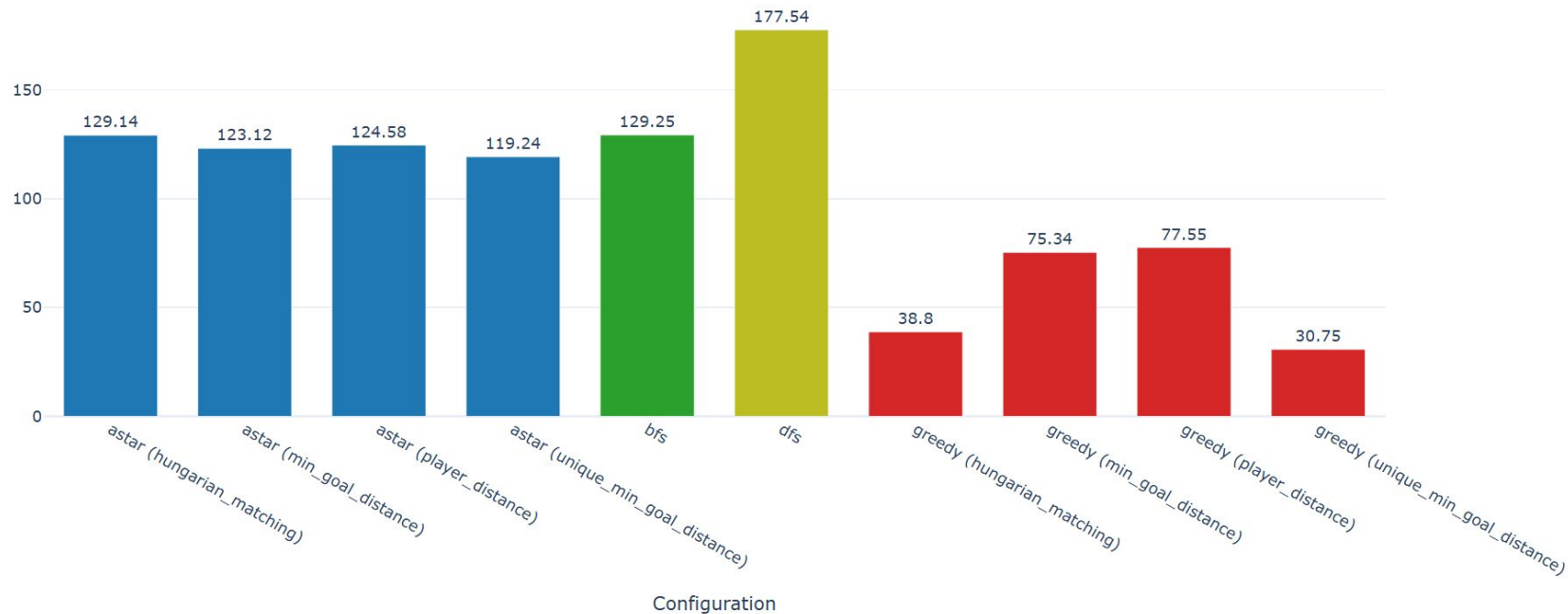


Costo de la solución y tiempo de ejecución

Análisis de eficiencia de algoritmos y heurísticas

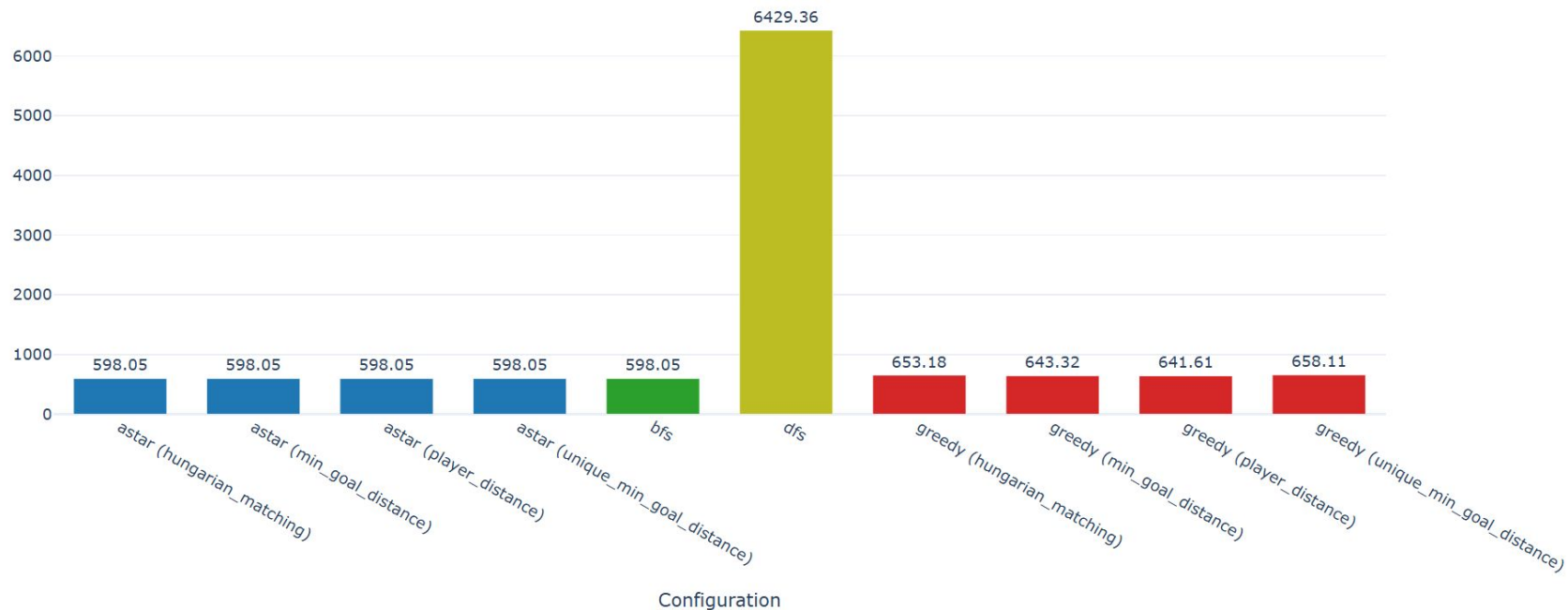


Resultados - Tiempo promedio por algoritmo



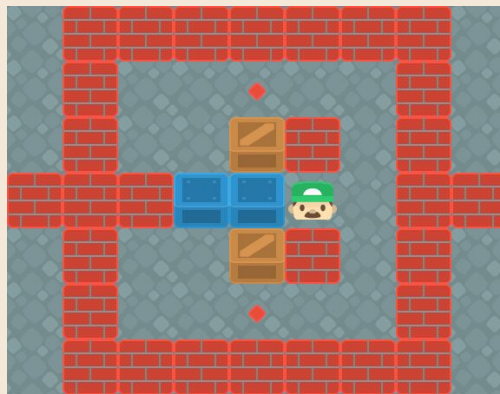
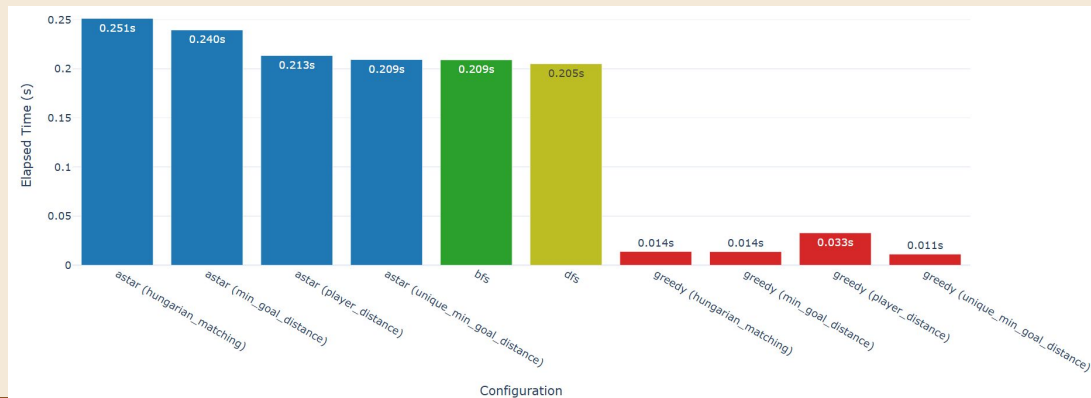
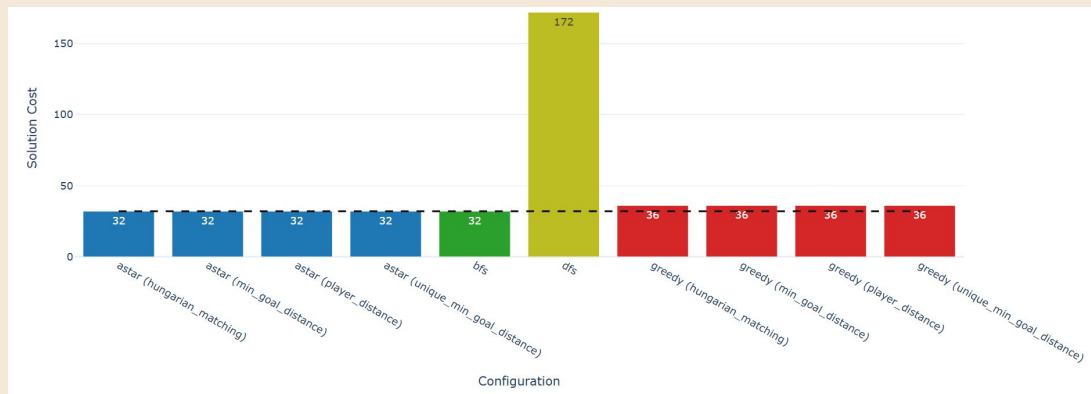


Resultados - Costo promedio por algoritmo

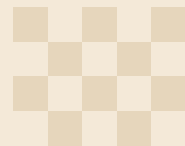




Nivel 22

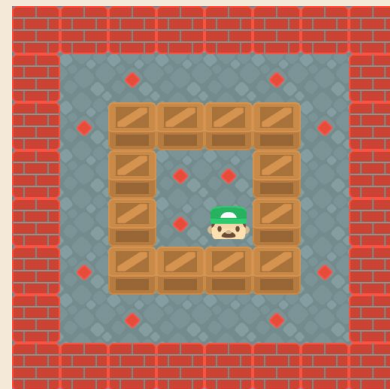
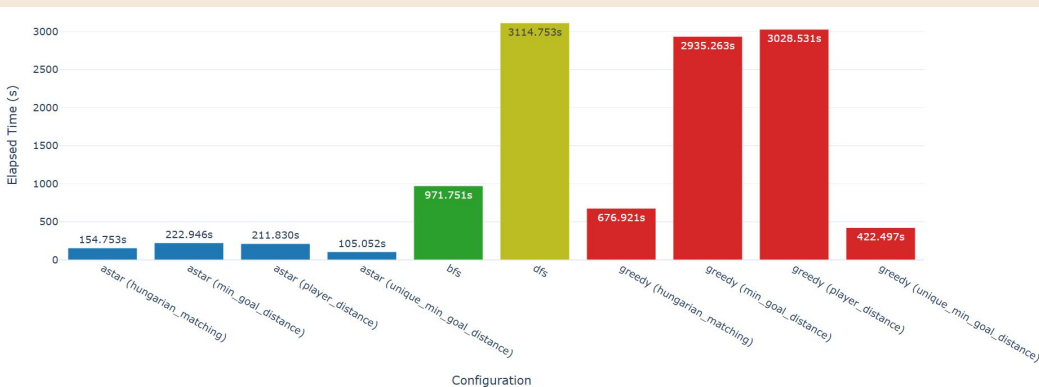
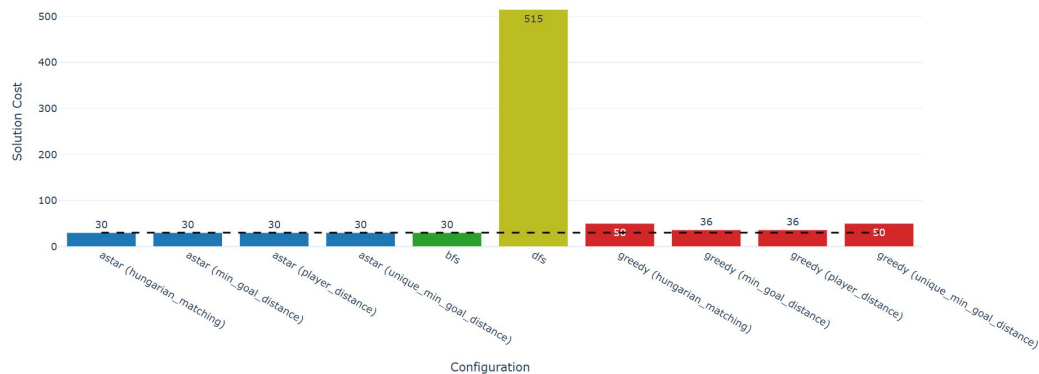


Oamino óptimo
BFS

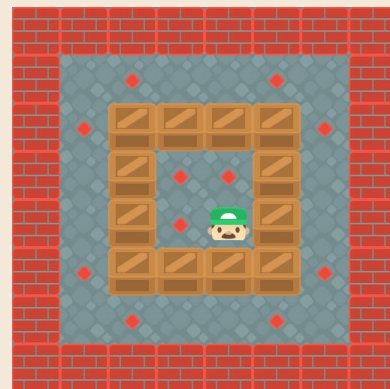




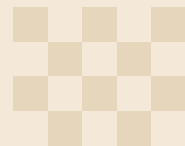
Nivel 8 (12 cajas)



Oamino óptimo
BFS

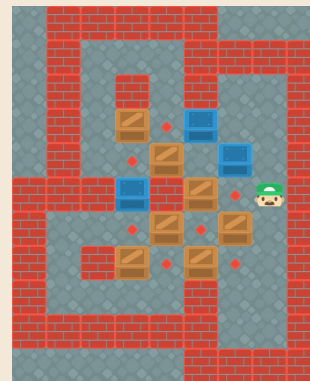
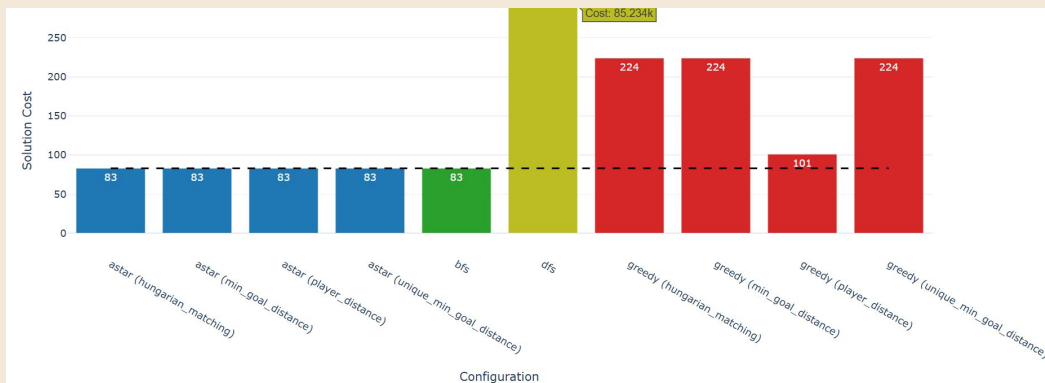


Greedy
Húngaro

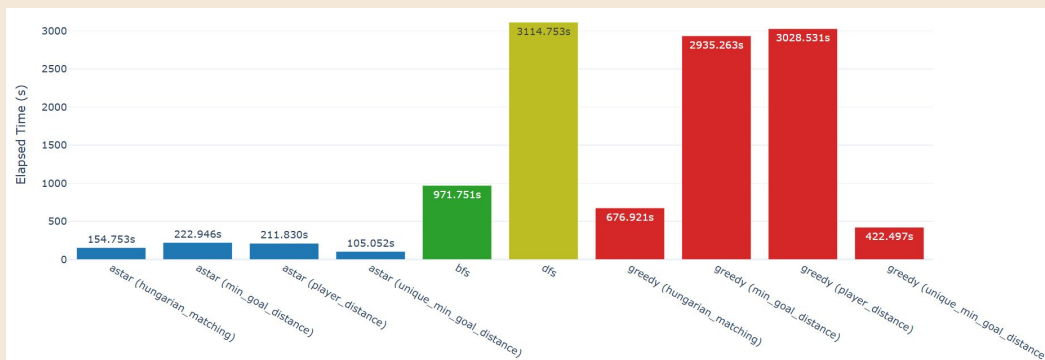




Nivel 79 (10 cajas)

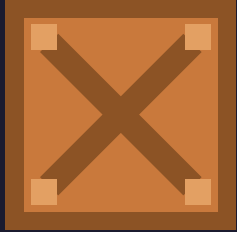


Camino óptimo
BFS



¿A qué se debe el disparo de
movimientos con DFS?

- ❑ DFS no garantiza camino óptimo
- ❑ “Libertad” de movimiento
- ❑ DFS elige una dirección fija

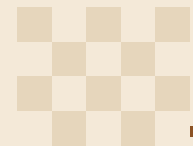
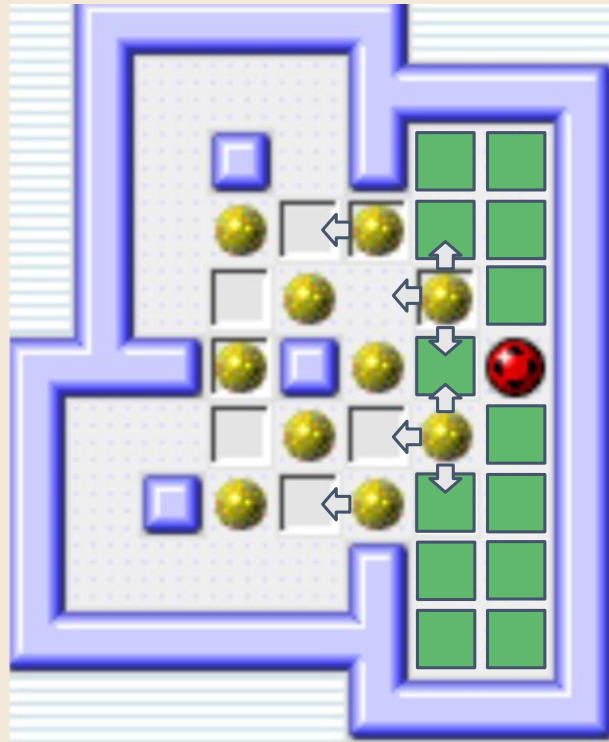


Optimización: Simplificación de estados equivalentes.

Colapso de estados

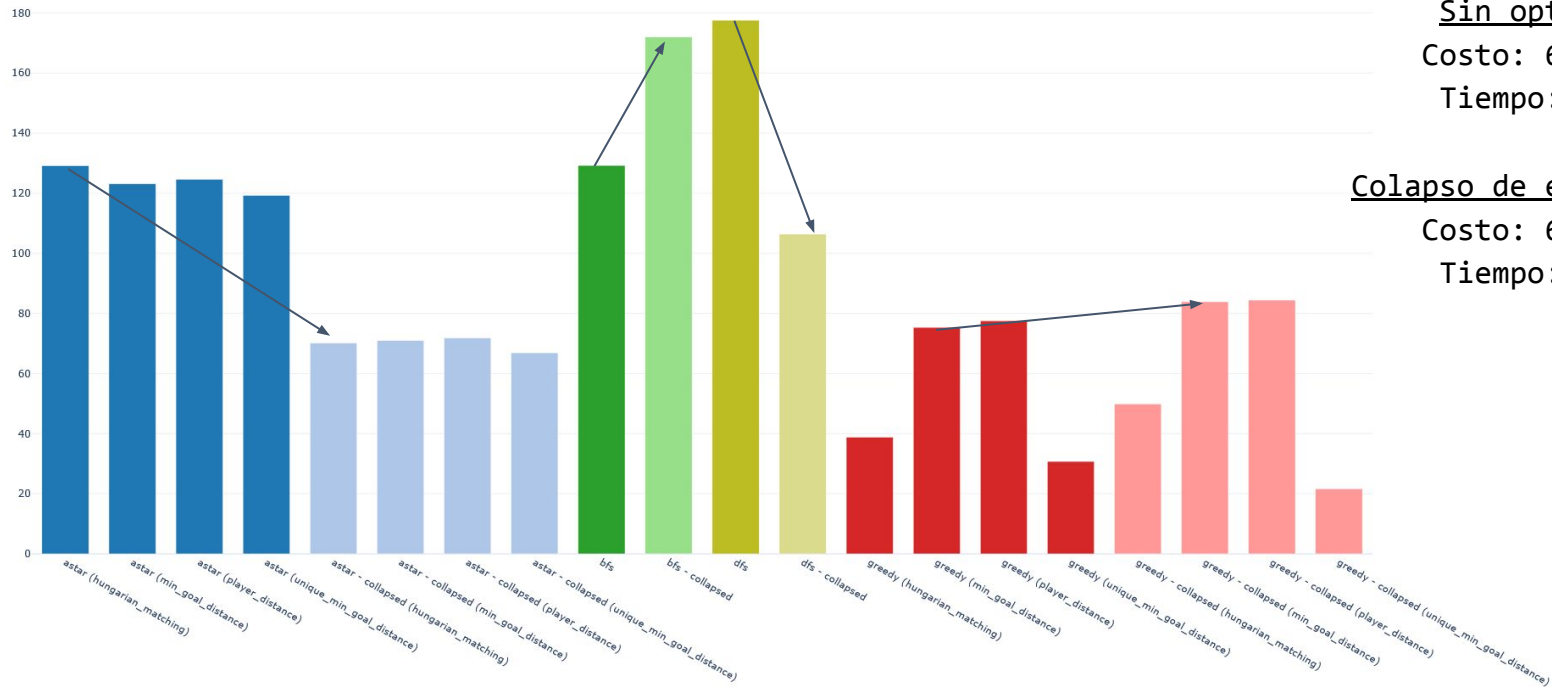


Colapso de estados





Resultados - Tiempo promedio por algoritmo



Sin optimizar

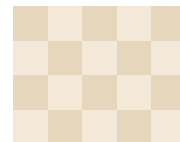
Costo: 672.887

Tiempo: 15,9h

Colapso de estados

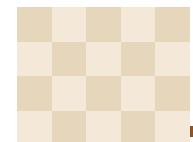
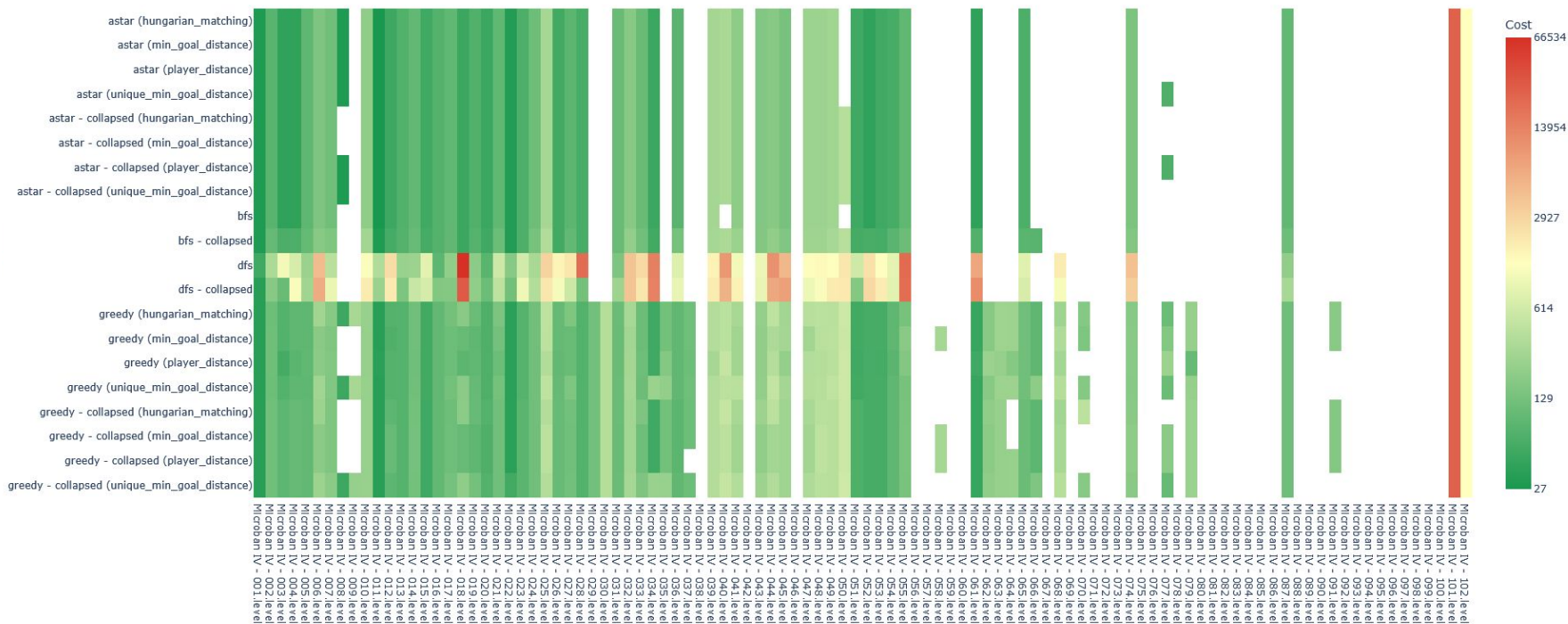
Costo: 675.124

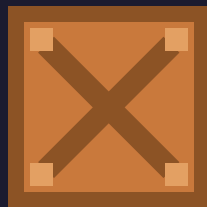
Tiempo: 12,4h





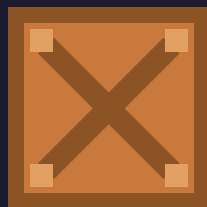
Resultados - Cost Heat Map





Encontrando la definición de *Dificultad*

Métricas para definir complejidad

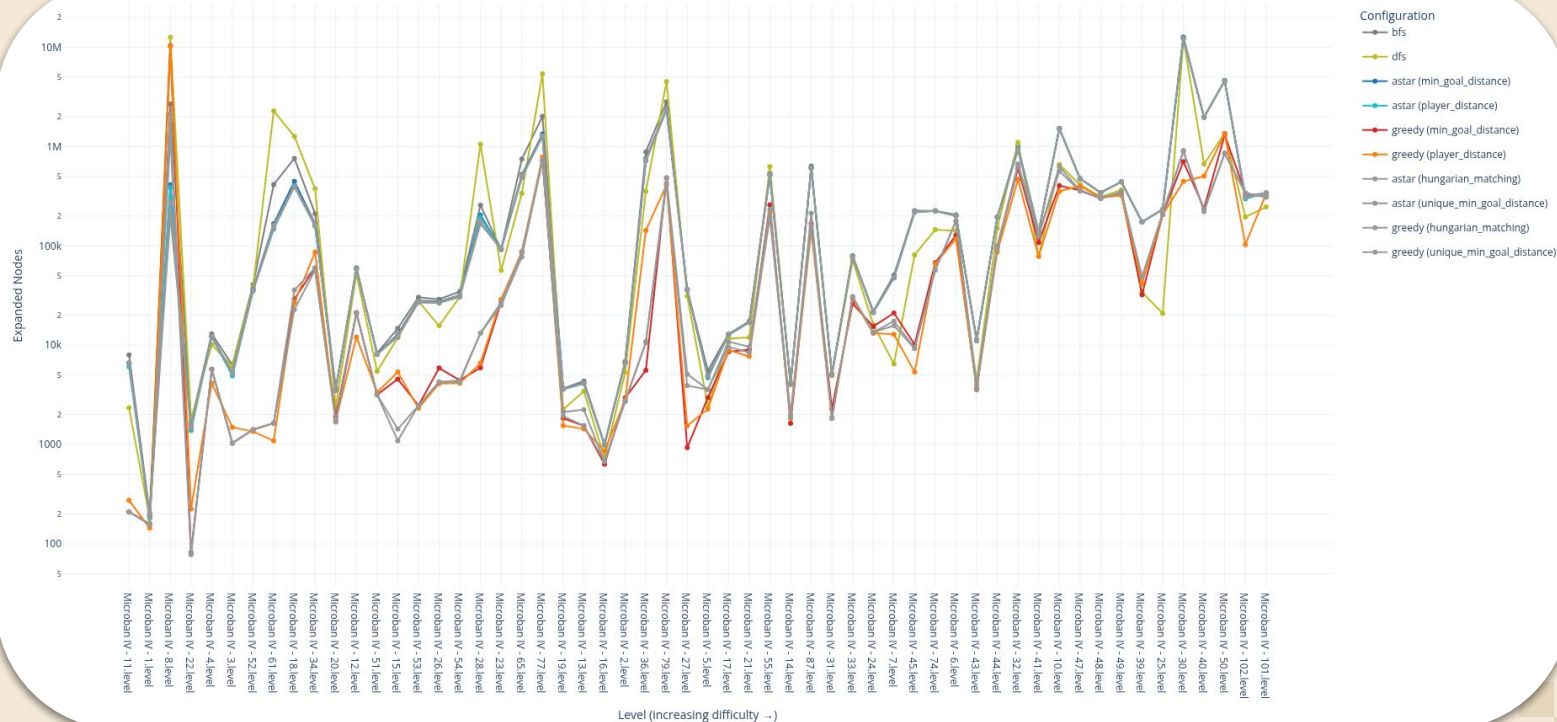


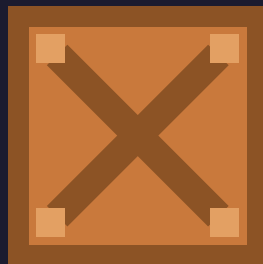
Escalabilidad

¿Mayor cantidad de movimientos implica mayor dificultad?

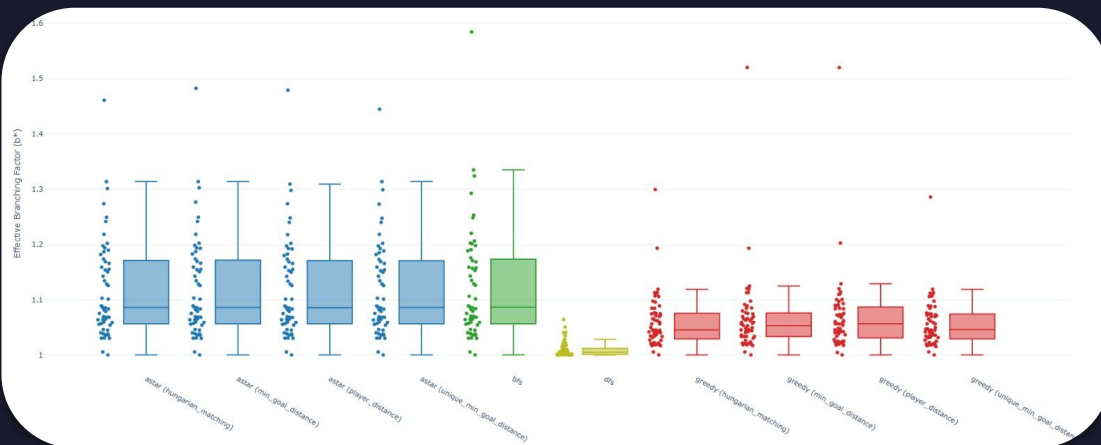


Niveles ordenados por "Dificultad" (costo)

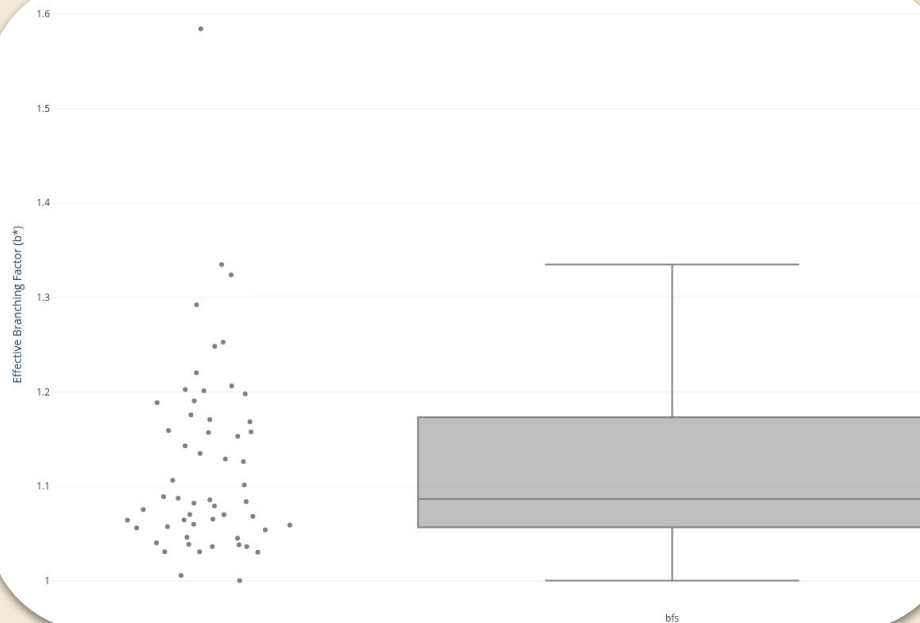




EBF

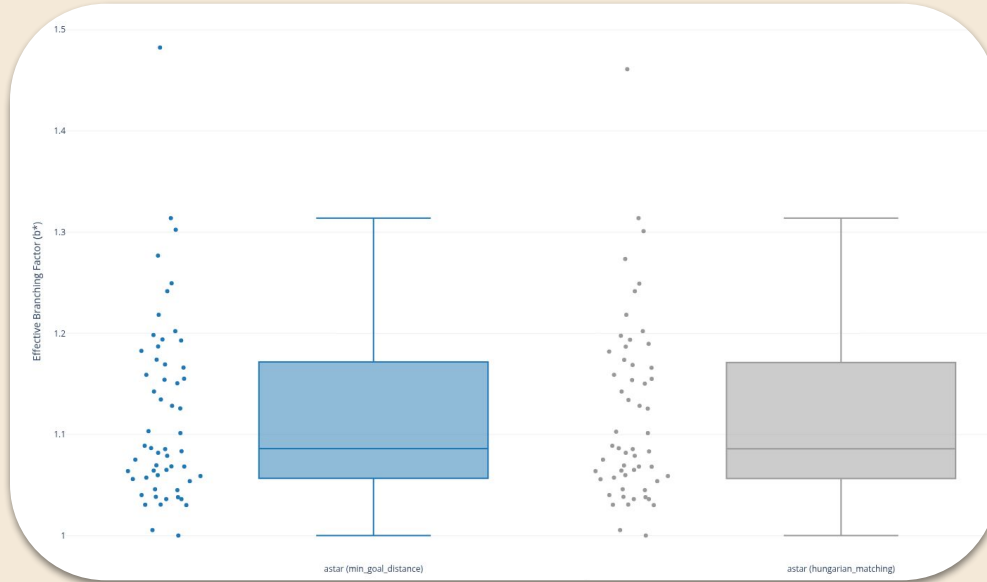


BFS



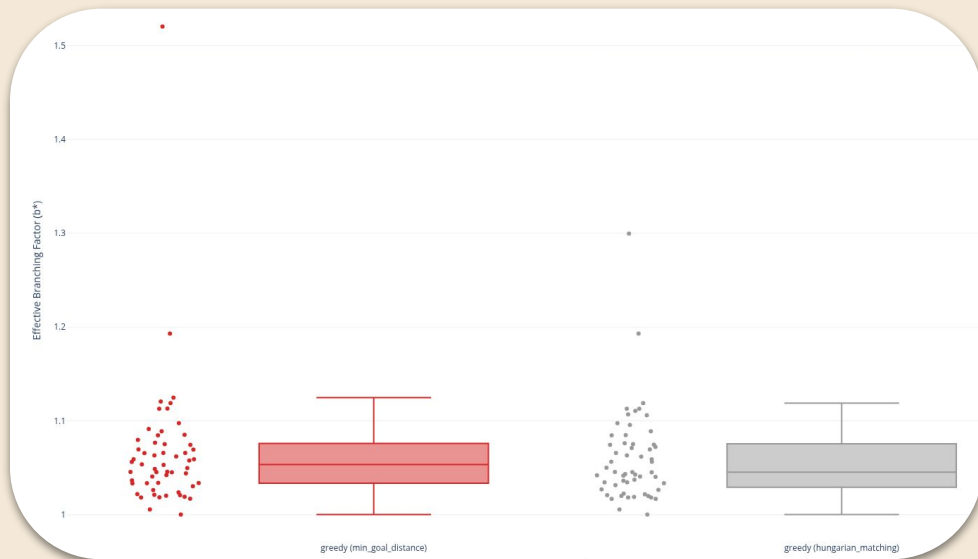
- ❑ Al no usar heurísticas, explora todas las ramas de manera uniforme en anchura
- ❑ Ineficiente para instancias profundas

A*



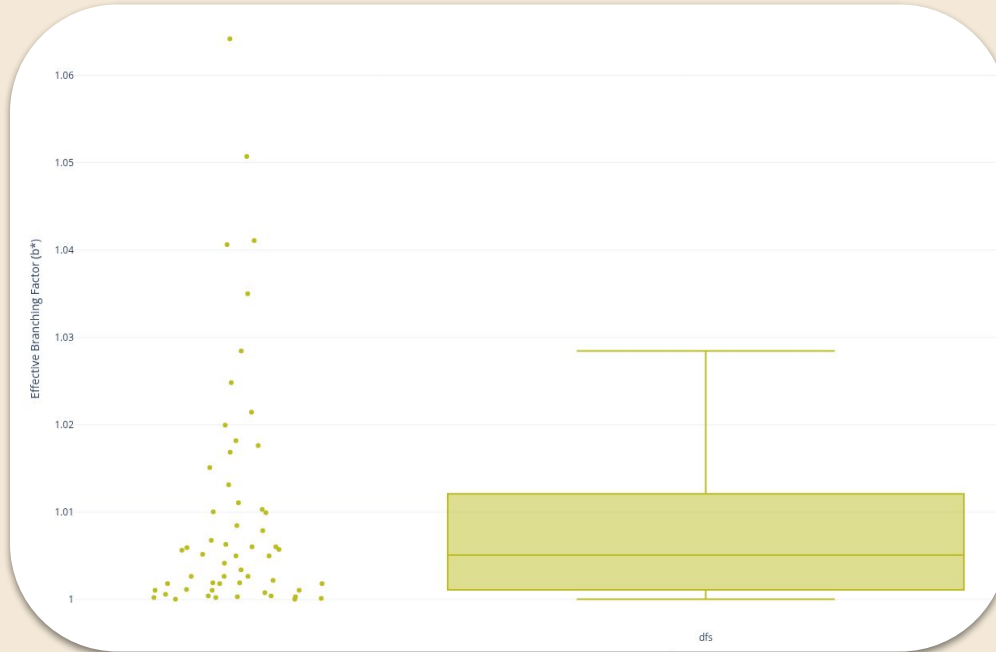
- ❑ Modestamente más bajo que con fuerza bruta
- ❑ La mejor combinación entre encontrar la ruta más corta y contener el crecimiento de nodos.

Greedy



- ❑ EBF bajo
- ❑ Muy rápido para encontrar salidas, pero sacrifica la ruta óptima.

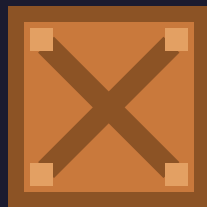
DFS



- ❑ Converge a 1
- ❑ **NO** significa que el algoritmo sea más eficiente
 - ❑ Se pierde en la profundidad



Resuelto en 4 min

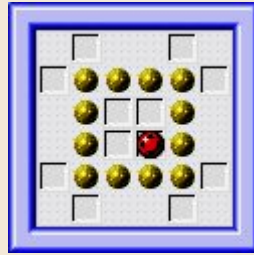
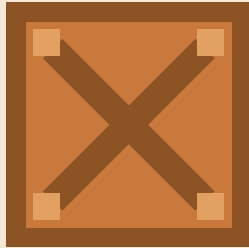


Resuelto en 11 sec

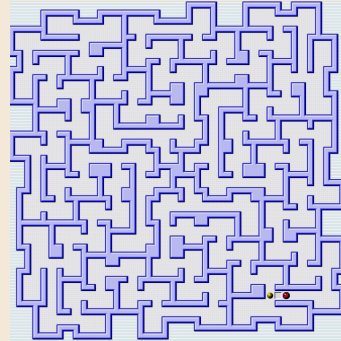
¿Que hace a un nivel difícil?

Análisis "preventivo"

Planteo



$\frac{132461 \text{ nodos}}{30 \text{ mov.}} = 4415.36$
EBF = 1.984



$\frac{246134 \text{ nodos}}{23930 \text{ mov.}} = 9.81$
EBF = 1.000

¿Solución? **EBF & expanded/solution**

- Permite saber de antemano que tan difícil será un nivel para un humano

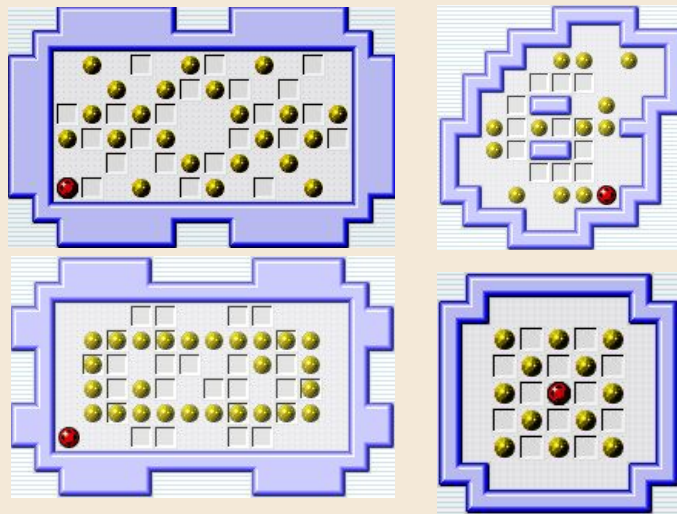
Esto puede definir “cuánto le **costó**”. *PASADO*

pero...

¿Como un humano puede enterarse de antemano cuánto le va a **costar** a la IA?



Niveles NO resueltos por falta de memoria



¿Que tienen en común?

- ❑ Alta cantidad de cajas en lugares muy compactados
 - ❑ El algoritmo húngaro **NO SIRVE**
 - ❑ Difícil de definir una **heurística**?
- ❑ Fácil de generar un freeze y un corral deadlock
- ❑ Patrones repetitivos y simétricos

